IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 6 June 2025, accepted 17 June 2025, date of publication 20 June 2025, date of current version 27 June 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3581957

RESEARCH ARTICLE

Fully Quantized Matrix Arithmetic-Only BERT Model and Its FPGA-Based Accelerator

HIROSHI FUKETA, (Member, IEEE), TOSHIHIRO KATASHITA, YOHEI HORI, (Member, IEEE), AND MASAKAZU HIOKI, (Member, IEEE)

National Institute of Advanced Industrial Science and Technology (AIST), Tsukuba, Ibaraki 305-8568, Japan

Corresponding author: Hiroshi Fuketa (h-fuketa@aist.go.jp)

This work was supported in part by JSPS KAKENHI under Grant JP24K02918.

ABSTRACT In this paper, we propose a fully quantized matrix arithmetic-only BERT (FQ MA-BERT) model to enable efficient natural language processing. Conventionally, the BERT model relies on floating point arithmetic for inference and requires not only linear matrix multiplication but also nonlinear functions, such as softmax and normalization functions, which significantly increase hardware costs. In contrast, the proposed FQ MA-BERT model quantizes all activations and weights to 8-bit integer precision and ternary precision, respectively, i.e., the proposed model is fully quantized. Moreover, all nonlinear layers are replaced with matrix arithmetic operations. This means that the proposed model consists solely of integer-precision linear matrix arithmetic operations, allowing a significant reduction in hardware resources. To validate the hardware-friendliness of the proposed FQ MA-BERT model, we implement an accelerator for the proposed model on AMD Zynq Ultrascale+ FPGA. The proposed accelerator comprises only two types of integer-precision matrix multiplication units without the need for specialized circuits to perform nonlinear functions. The implementation results show that the hardware resource efficiency of the proposed accelerator is 1.8 times higher than that of conventional FPGA-based BERT accelerators.

INDEX TERMS BERT, FPGA, natural language processing, quantization, transformer.

I. INTRODUCTION

The BERT (bidirectional encoder representations from transformers) models are widely utilized for natural language processing tasks, including language inference and question answering [1]. However, its model size and computational complexity are significantly large [2], which makes it difficult to use BERT model for AI applications on mobile devices, commonly referred to as edge AI [3]. Therefore, a lightweight model suitable for edge AI is necessary.

In edge AI, only inference is usually performed. In this case, a lightweight model with reduced memory and computational requirements can be realized. In this paper, we define such a model as a ''hardware-friendly'' model. The requirements are as follows:

- **Fully quantized:** Quantized weights in a model can reduce not only the memory footprint but also the bandwidth. Additionally, quantized activations can reduce the energy consumption required for computation while enhancing performance per area. A model in which both weights and activations are quantized is referred to as a fully quantized model. Extremely low-bit quantization, such as binary and ternary precision, is particularly advantageous.
- **Uniformity of calculation:** When model's computations consist entirely of uniform calculations, such as matrix multiplication, hardware resource utilization can be maximized, leading to higher energy efficiency and improved performance per area.
- **Easy operations:** In terms of energy efficiency and performance, addition, subtraction, and shift operations are preferable to more complex operations such as multiplication, division, and nonlinear operations. Table lookups are also easily implementable in hardware. Thus, such complex computations in the model should be replaced with hardware-friendly operations.

The associate editor coordinating the review of this manuscript and approving it for publication was Ludovico Minati.

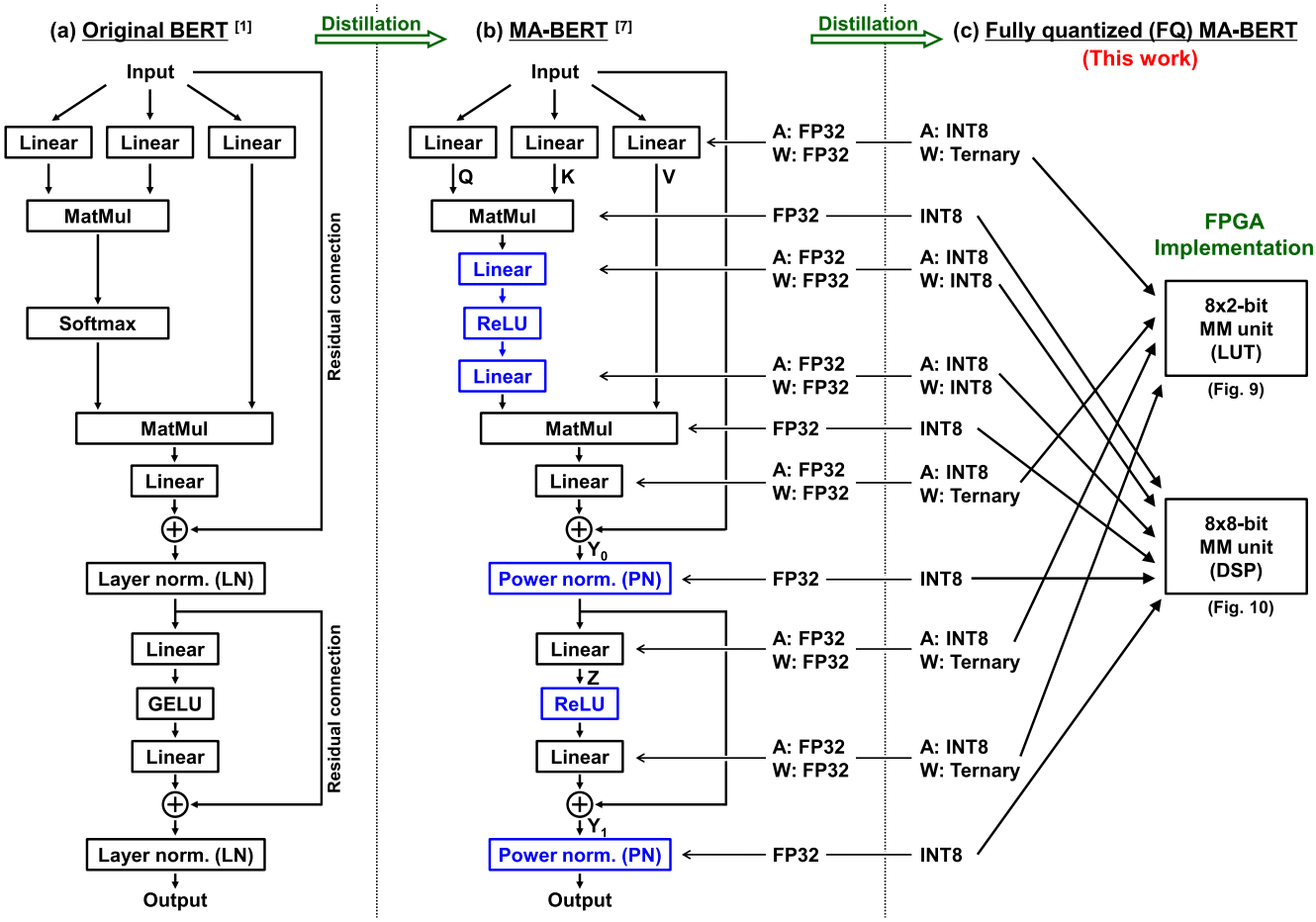

FIGURE 1. Architecture of the transformer block in (a) the original BERT, (b) the matrix arithmetic-only BERT (MA-BERT), and (c) the proposed fully quantized (FQ) MA-BERT. The proposed FQ MA-BERT model consists solely of linear matrix arithmetic operations. Therefore, its accelerator can be implemented using only two types of integer-precision matrix multiplication (MM) units. The detailed implementation is described in Section III.

Various lightweight BERT models have been proposed. For instance, TernaryBERT [4] and BinaryBERT [5] quantize weights to ternary and binary precision, respectively. In contrast, activations in the linear layers are quantized to 8-bit integer (INT8) precision, whereas activations in the nonlinear layers, such as softmax and layer normalization (LN) layers, remain unquantized. In this respect, these models are not fully quantized. I-BERT, introduced in [6], was presented as a fully quantized BERT model. In the I-BERT model, weights are quantized to INT8 precision and activations are also INT8-quantized not only in the linear layers but also in the nonlinear layers. However, complex computations are required for the nonlinear layers.

The above conventional studies have highlighted the difficulties associated with computing nonlinear layers in the BERT model. To address this issue, MA-BERT was proposed in [7]. In MA-BERT, computations in the softmax and LN layers are replaced with matrix-matrix multiplication and element-wise matrix multiplication. In addition, GELU layers are substituted with ReLU layers. This means that the MA-BERT model consists exclusively of matrix arithmetic operations, achieving the highest uniformity of calculation. However, quantization is not considered.

Other hardware-friendly models, in which multiplication operations are substituted with table lookups, were proposed in [8], [9], and [10]. These models can achieve higher performance with lower power consumption when implanted in custom hardware such as ASIC and FPGA [9], [10]. However, when applied to BERT, they suffer from a decline in inference accuracy [8].

In this paper, we propose a fully quantized BERT model that consists exclusively of linear matrix arithmetic operations, building upon MA-BERT [7]. In the proposed model, the weights of the linear layers are quantized to ternary precision, while the activations are quantized to INT8 precision, i.e. the proposed model is fully quantized. Furthermore, we demonstrate an FPGA-based accelerator to show how the hardware-friendliness of the proposed model enhances FPGA resource efficiency.

Conventionally, various FPGA-based BERT accelerators have been demonstrated in [11], [12], [13], [14], and [15]. In FQ-BERT [11], NPE [12], and FET-OPU [13], activations

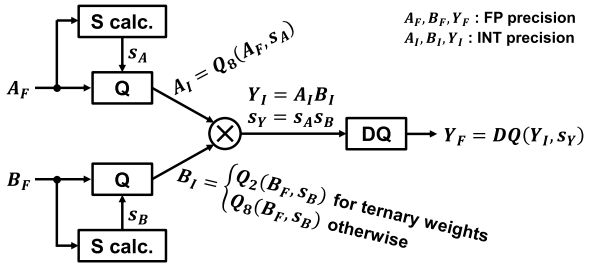

FIGURE 2. Conventional quantized matrix multiplication (MM). The input activations and weights, originally in floating-point (FP) precision, are quantized using the functions Q_8 and Q_2 defined in Eqs. (1) and (3), respectively. Their scaling factors are given by Eqs. (2) and (4). The multiplication is computed with integer (INT) precision. Then, the multiplication results are dequantized by the de-quantizer defined in Eq. (5). Thus, the outputs are FP precision.

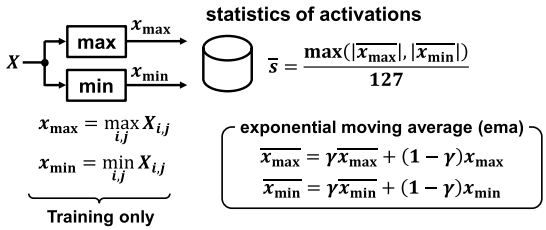

FIGURE 3. Static quantization introduced in I-BERT [6]. In this approach, the moving average scaling factors $\bar{s}$ are derived from the statistics of activations using an exponential moving average during the training phase. We set γ to 0.95 in this work.

and weights are quantized to INT8, and dedicated circuitry is required to compute nonlinear functions, such as softmax and LN layers. These computations are performed using a lookup table-based approach [11], [13] and piecewise linear approximation [12]. In contrast, TATAA [14] and BAT [15] rely on floating-point precision arithmetic for nonlinear functions, which degrades hardware resource efficiency. By comparison, the proposed accelerator consists solely of two types of integer-precision matrix multiplication units, eliminating the need for specialized circuits to perform nonlinear functions. Implementation results indicate that the resource efficiency of the proposed accelerator is 1.8 times higher than those of the conventional accelerators [11], [12], [13], [14], [15].

The remainder of this paper is organized as follows. The details of the proposed BERT model are described in Section II. The architecture of the FPGA-based accelerator and implementation results are presented in Section III and Section IV, respectively. Finally, Section V concludes this paper.

II. FULLY QUANTIZED MA-BERT MODEL

A. MODEL ARCHITECTURE

The BERT model ($\text{BERT}_{\text{BASE}}$) consists of 12 transformer blocks (or encoder layers) [1]. In this paper, we focus on these blocks, as they account for the majority of computations in BERT. Fig. 1(a) shows the architecture of the transformer block in the original BERT. Each transformer block consists of linear layers, matrix multiplication (MM) blocks, softmax layers, layer normalization (LN) layers, and a GELU layer.

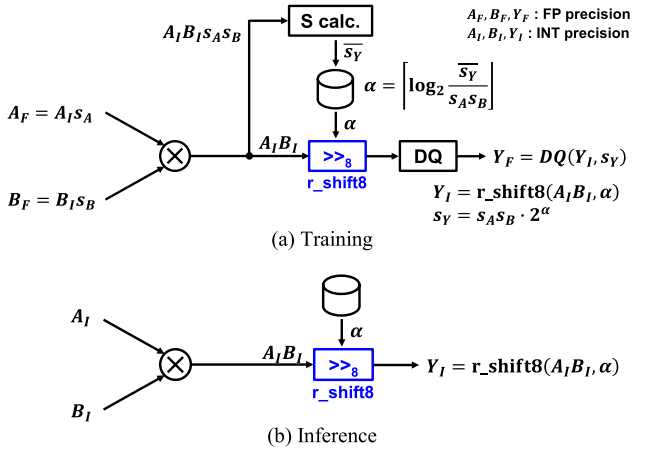

FIGURE 4. Proposed fully quantized matrix multiplication (MM) with shifter used in FQ MA-BERT. (a) During the training phase, the moving average scaling factors explained in Fig. 3 are derived from the multiplication results. The right shifter defined in Eq. (6) is used to adjust the scaling factors. α represents the amount of shift. (b) In the inference phase, the calculation of α is not required, and only INT precision multiplication and shift operations are performed.

As described in Section I, the nonlinear layers in the BERT model (softmax, LN, and GELU layers) reduce computational efficiency. Thus, MA-BERT was proposed in [7] to eliminate these nonlinear layers, as illustrated in Fig. 1(b). In the MA-BERT model, the softmax layer is replaced with a two-stage linear layer with a ReLU activation function. In addition, the LN layers are substituted with Power Normalization (PN), a Batch Normalization-like technique proposed in [16]. During inference, PN requires only element-wise matrix multiplication between the activations and PN parameters. Furthermore, the ReLU layer is used instead of the GELU layer in the MA-BERT model. As a result, the MA-BERT model consists exclusively of linear matrix arithmetic operations. However, these operations are performed using floating-point (FP) precision.

In this paper, we propose a fully quantized (FQ) MA-BERT model, as illustrated in Fig. 1(c). In FQ MA-BERT, the activations are quantized to INT8 precision. The weights of the linear layers are ternary quantized, while the parameters of the softmax (the weights of the two-stage linear layers) and PN layers are quantized to INT8 precision. Consequently, the proposed FQ MA-BERT requires an MM with two types of integer precision: 8×2 (ternary) bits and 8×8 bits. Assuming FPGA implementation, the MM with the former precision can be implemented using only LUTs (lookup tables), whereas the MM with the latter precision can be implemented using DSP slices. The implementation details are discussed in Section III.

B. QUANTIZATION TECHNIQUES

As shown in Fig. 1(c), the proposed FQ MA-BERT model is quantized using INT8 and ternary precision. The quantization operation to INT8 precision is expressed as,

$$Q_8(X, s) = \text{clamp}\left(\left\lceil \frac{X}{s} \right\rfloor, -127, 127\right), \quad (1)$$

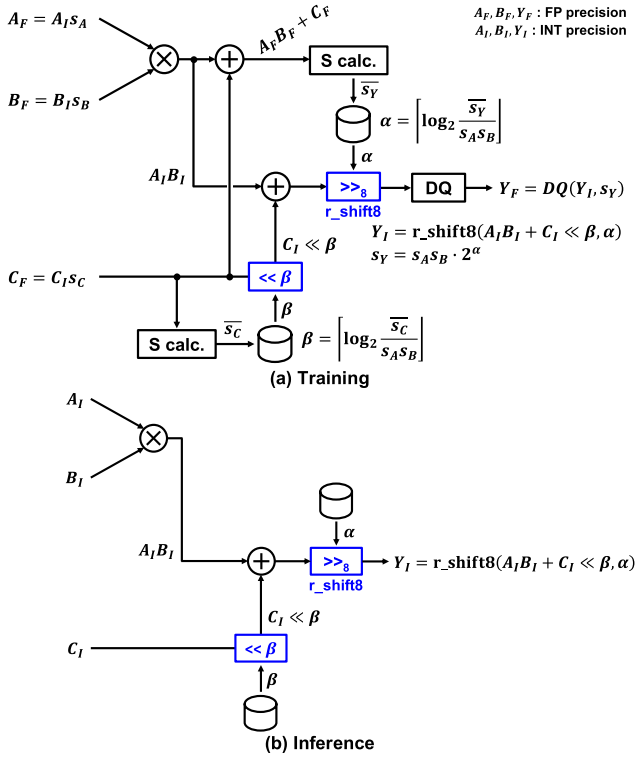

FIGURE 5. Proposed fully quantized matrix multiplication (MM) withshifter used in FQ MA-BERT for residual connections in (a) training and(b) inference. A left shifter is employed to adjust the scaling factor of theresidual connection.

TABLE 1. Accuracy of sequence classification on the glue benchmark.

Model	Precision	Accuracy (GLUE benchmark)		
		CoLA	MNLI	RTE
Original BERT [1]	FP32	55.8	84.5 / 84.7	67.1
MA-BERT [7]	FP32	58.6	84.7 / 84.5	68.2
FQ MA-BERT (This work)	Ternary / INT8	52.9	83.1 / 83.0	69.7

where X is a matrix and $\lceil \cdot \rfloor$ represents the rounding operation. s is a scaling factor of X and is given by,

$$s = max \left| X_{i,j} \right|. \tag{2}$$

In contrast, the quantization operation to ternary precision is expressed as,

$$Q_2(X, s) = \begin{cases} X_{i,j} = 1 \text{ if } X_{i,j} > s \\ X_{i,j} = -1 \text{ if } X_{i,j} < -s \\ X_{i,j} = 0 \text{ otherwise.} \end{cases} \tag{3}$$

where s is given by the following equation according to [4], [17],

$$s = \frac{0.7}{NM} \sum_{i}^{N} \sum_{j}^{M} \left| X_{i,j} \right| \tag{4}$$

Fig. 2 illustrates a conventional quantized MM. The activations for all layers, as well as the parameters for the

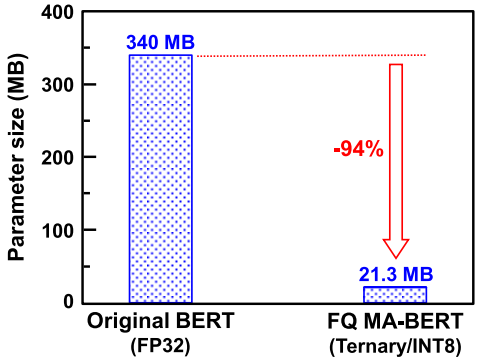

FIGURE 6. Comparison of parameter sizes of transformer blocks (Fig. 1)between the original BERT model [1] and the proposed FQ MA-BERTmodel.

TABLE 2. Comparison of lightweight bert models.

Model	Parameter Size*1 (MB)	Accuracy (GLUE benchmark)			
		CoLA	MNLI	RTE	Avg.
DistilBERT [22]	170	51.3	82.2	59.9	64.5
TinyBERT$_6$ [23]	170	51.1	84.6	70.0	68.6
TinyBERT$_4$ [23]	18	44.1	82.5	66.6	64.4
TernaryBERT [4]	21	50.1	83.1	68.9	67.4
FQ MA-BERT (This work)	21	52.9	83.1	69.7	68.6

(*1) Parameter size of transformer block (Fig. 1).

softmax and PN layers, are quantized using Eq. (1), while the weights of the liner layers are quantized using Eq. (3). The corresponding scaling factors derived from Eqs. (2) and (4), respectively. The multiplication results are then de-quantized as,

$$DQ(X, s) = s \cdot X. \tag{5}$$

This conventional quantized MM has the following two drawbacks in terms of hardware-friendliness: 1) The scaling factors of the activations must be always (in both training and inference phase) computed before quantization, and 2) The input and output values of the multiplication remain in FP precision, that is, the model is not fully quantized.

To address these drawbacks, we introduce a static quantization technique used in I-BERT [6]. In static quantization, the maximum and minimum values of the input activations are computed only during the training phase. Using the exponential moving average of them, a moving average scaling factor is calculated and stored, as shown in Fig. 3. During inference, the stored moving average scaling factors are used instead of Eq. (2), which means that the scaling factor calculation is not required.

In I-BERT [6] and FQ-BERT [11], the output scaling factors of the MM are adjusted, which requires additional multiplication operations with high computational precision. Thus, hardware costs are increased. To address this issue, we propose a shift-based scaling factor adjustment technique, as illustrated in Fig. 4. In the proposed approach, shift

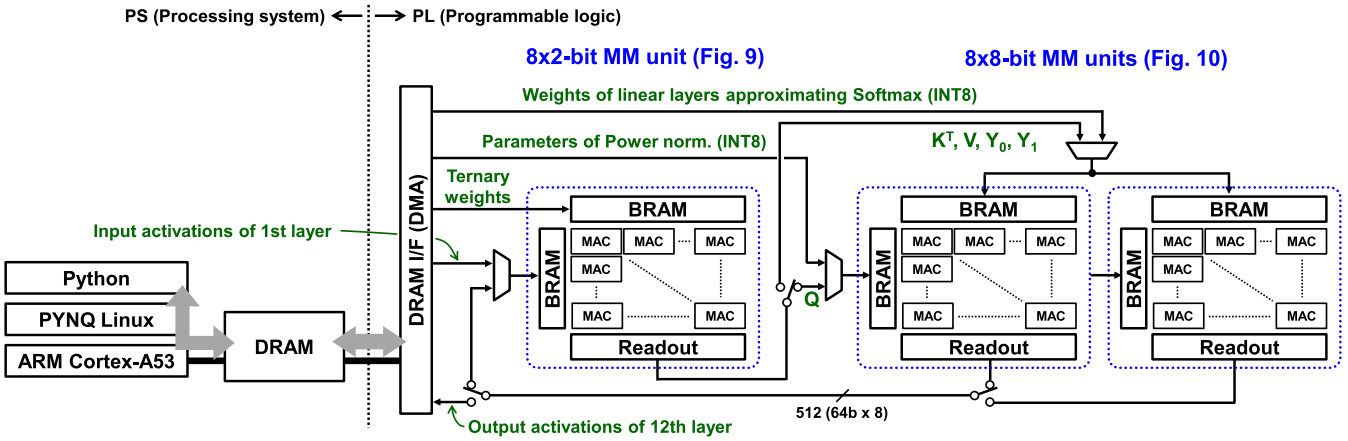

FIGURE 7. Overall architecture of the proposed accelerator for the FQ MA-BERT model (Fig. 1). The activations (matrices) Q, K, V, Y0, Y1 are defined in Fig. 1.

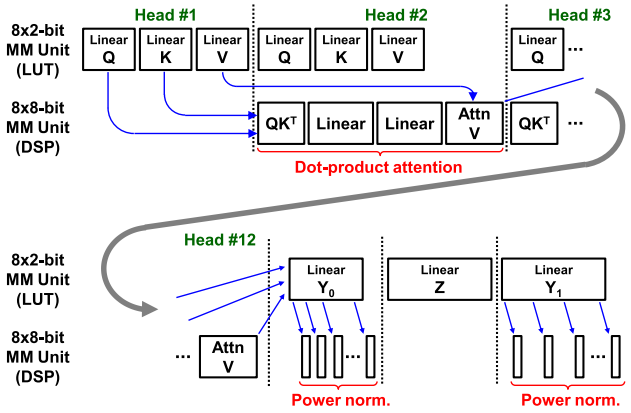

FIGURE 8. Dataflow of the proposed FQ MA-BERT accelerator.

operations replace multiplication operations, because shift operations are more hardware-friendly. During the training phase, the shift amount (α in Fig. 4) is determined based on the scaling factors of the input values and their multiplication results, as shown in Fig. 4(a). After that, the INT8-limited right shift operation defined in the following equation is performed using α to adjust the scaling factors,

$$\text{r\_shift8}\,(X, \alpha) = \text{clamp}\,(X \gg \alpha, -127, 127) \qquad (6)$$

In the inference phase, the MM in the proposed FQ MA-BERT model consists solely of INT precision multiplication and shift operations. Please note that α is calculated based on statistics collected during the training phase, as shown in Figs. 3 and 4(a); therefore, the calculation of α is not required during inference.

The BERT model includes residual connections, as illustrated in Fig. 1. These residual connections require additional scaling factor adjustments. In this work, a left-shift operation is employed for the residual connection, as shown in Fig. 5. The shift amount (β in Fig. 5) is computed using the moving average of the scaling factor in the residual connection during the training phase (Fig. 3). Consequently, the proposed MM with a residual connection can be performed with INT precision multiplication, addition, and shift operations during the inference phase, as shown in Fig. 5(b).

C. TRAINING PROCEDURE

Quantization-aware training (QAT) is essential to achieve extremely low-bit quantization, such as ternary precision [4]. In this paper, a three-step training is conducted. In the first step, we utilize the pre-trained model of bert-base-uncased model [18] and fine-tune it for sequence classification on the GLUE benchmark [19]. For fine-tuning, the learning rate is 2e-5, the batch size is 32, and the number of training epochs is 3, according to [20]. In the second step, the MA-BERT model is fine-tuned using knowledge distillation [21]. In the training, the teacher is the fine-tuned BERT model from the first step, and the student model is initialized with the pre-trained MA-BERT model provided by the developers of MA-BERT. The hyperparameters for this step follow the configuration described in [7]. Finally, in the third step, the fine-tuned MA-BERT model is quantized to obtain the proposed FQ MA-BERT model through knowledge distillation as well.[1] The weights of the student (FQ MA-BERT) are initialized with the parameters fine-tuned in the second step and trained using 32-bit floating point (FP32) precision, as shown in Figs. 4(a) and 5(a). The hyperparameters follow the settings described in [4], with the exception that the number of training epochs is set to 3 for all tasks in the GLUE benchmark.

The inference accuracies for sequence classification on the GLUE benchmark are listed in Table 1. The accuracy of the MA-BERT model trained in the second step is identical to that of the original BERT model trained in the first step. This means that the approximations of the GELU, softmax, LN layers (Fig. 1(b)) have a negligible impact

[1]The trained models are released at https://doi.org/10.57765/2003355

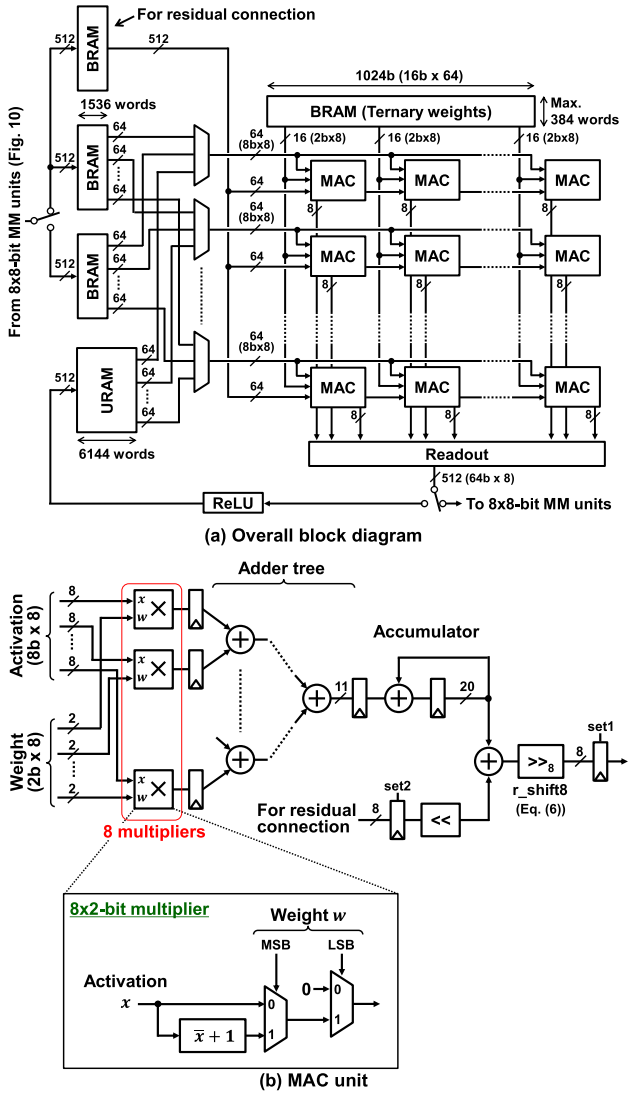

FIGURE 9. Architecture of 8 × 2-bit matrix multiplication (MM) unit. ThisMM unit is implemented on LUTs.

TABLE 3. Utilization of pl when the proposed FQ MA-Bert accelerator is implemented on ZCU104 (XCZU7EV FPGA).

Resource	Available	Used	Utilization (%)
LUT	230,400	152,490	66.2
FF	460,800	136,677	29.7
BRAM	312	188.5	60.4
URAM	96	16	16.7
DSP slice	1,728	1,024	59.3

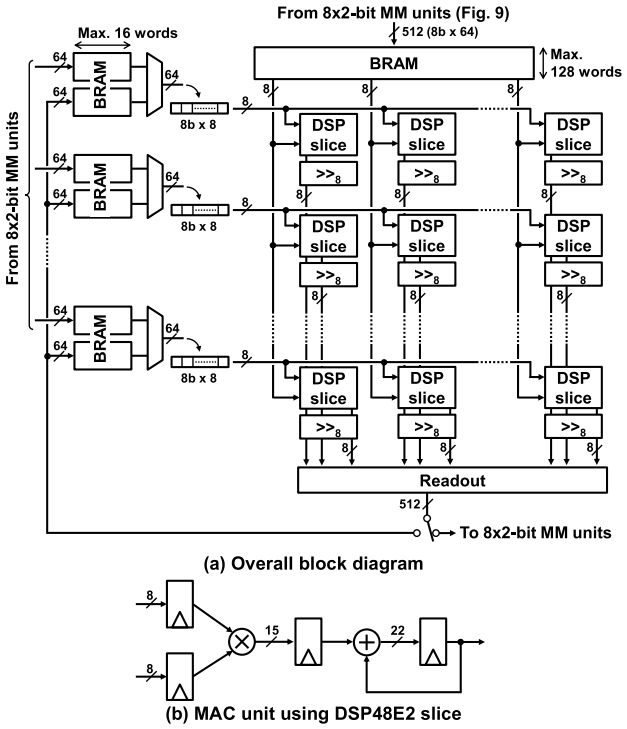

FIGURE 10. Architecture of 8x8-bit matrix multiplication (MM) unit. ThisMM unit is implemented using DSP slices. ">>8" represents the INT8-limited right shifter defined in Eq. (6).

TABLE 4. Performance summary.

Theoretical peak performance	
INT8/2 MM x 1 unit (LUT)	1,581 GOPS
INT8/8 MM x 2 units (DSP)	395.2 GOPS
Total	1976 GOPS

Actual (average) performance	
# of MACs	11.78 GMACs
Latency	16.35 ms
Performance	1441 GOPS

1 MAC = 2 OPs

on accuracy. In contrast, the accuracy of the proposed FQ MA-BERT model trained in the third step shows a slight degradation for the CoLA task, while the accuracies for the MNLI and RTE tasks are almost the same or even better. This accuracy degradation is caused by quantizing the weights to ternary precision and the activations to INT8 precision (Fig. 1(c)). Fig. 6 compares the model sizes of transformer blocks (Fig. 1) in the original BERT and the proposed FQ MA-BERT models. Owing to the quantization of weights to ternary precision, the model size is reduced by 94%. Table 1 and Fig. 6 demonstrate that the proposed FQ MA-BERT achieves a significant reduction in memory size with a modest degradation in accuracy. Table 2 shows a comparison of the proposed FQ MA-BERT model with the conventional lightweight BERT models [4], [22], [23]. The FQ MA-BERT model achieves the highest accuracy despite its smaller size. In addition, the conventional models require FP32 precision, whereas the FQ MA-BERT model only requires INT8 precision. This means that the proposed FQ MA-BERT model is advantageous in terms of hardware-friendliness.

III. FPGA-BASED ACCELERATOR IMPLEMENTATION

A. OVERALL ARCHITECTURE

In Section II, the FQ MA-BERT model was proposed as a hardware-friendly BERT model. In this paper, an accelerator

TABLE 5. Comparison of FPGA-Based Bert accelerators.

		FQ-BERT [11]	NPE [12]	FET-OPU [13]	TATAA [14]	BAT [15]	This work
FPGA platform		ZCU102	VCU118	Alveo U280	Alveo U280	ZCU102	ZCU104
Data format	MatMul	INT8 / INT4	INT8	INT8	INT8	INT4 / Binary	INT8 / Ternary
	Nonlinear func.	FxP8 *1	FxP16 *1	FxP32 *1	BF16	FP16	Not required
Frequency (MHz)		214	200	200	225	200	193
FPGA utilization	LUT (k)	124.4	192.4	886.8	724.9	146.5	152.5
	FF (k)	123.2	351.1	716.6	1154.9	137.0	136.7
	BRAM	419	369	1357	1472	114	188.5
	DSP	1751	2018	4864	4352	1024	1024
GLUE benchmark	CoLA	N.A.	N.A.	N.A.	N.A.	49.7	52.9
	MNLI	81.1 / 80.4			N.A.	83.2 / 83.3	83.1 / 83.0
	RTE	N.A.			N.A.	67.1	69.7
	SST-2	91.5			92.3	92.4	N.A.
Power (W)		9.8	20.0	7.4	10.8	5.5	9.7
Latency (ms)		43.9	15.0 *2	6.8	8.6	N.A.	16.4
Performance (TOPS)		N.A.	N.A.	1.64	2.94	1.12	1.44

(*1) FxPn represents n-bit fixed-point format. (*2) Sequence length is 128.

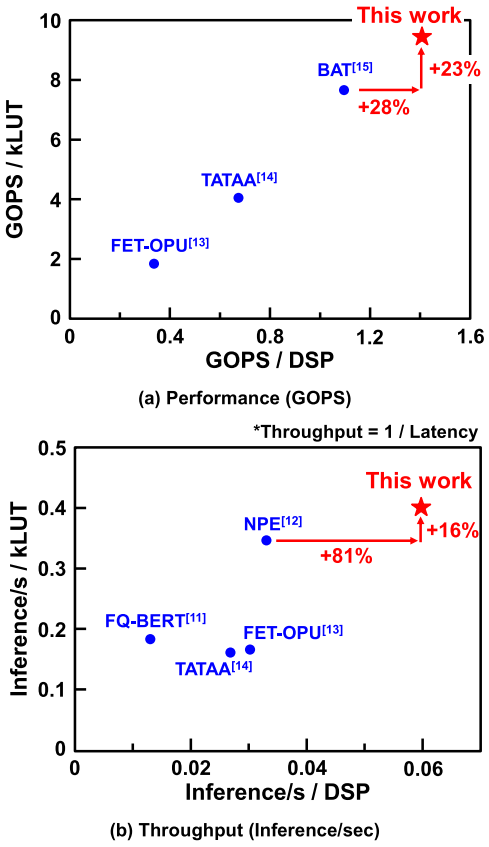

FIGURE 11. Comparison of resource efficiency, which is defined as theperformance and throughput normalized by FPGA utilization.

for the model is implemented on an FPGA to demonstrate how hardware-friendliness of the model can improve hardware resource efficiency. We use the ZCU104 evaluation board [24], which contains a Zynq UltraScale+ XCZU7EV MPSoC FPGA chip and a 2 GB DDR4 DRAM. The FPGA chip comprises an ARM Cortex-A53, called processing system (PS), and programmable logic (PL). As illustrated in Fig. 7, the proposed accelerator for the transformer blocks shown in Fig. 1 is implemented on the PL, and the PYNQ platform is used for ease of development [10]. The weights, parameters, and input/output activations of the transformer blocks (Fig. 1) are transferred via DRAM shared between the PL and PS. This data transfer is controlled by a Python program running on the PYNQ platform.

The proposed accelerator consists of two types of integer-precision MM units, as shown in Fig. 7. The first unit is based on 8 × 2 (ternary) bit multiplication and accumulation (MAC) and is implemented on LUTs only. The second unit is based on 8 × 8 bit MAC and is implemented using DSP slices. Fig. 8 shows the dataflow of the proposed accelerator. In the BERT model, the activations (matrices) Q, K, and V defined in Fig. 1 are divided into 12 sub-matrices (heads), called multi-head attention [25]. In the proposed accelerator, the linear layers for generating Q, K and V and the dot-product attention (matrix multiplication and softmax) layers are computed simultaneously on 8 × 2-bit and 8 × 8-bit MM units, respectively, as illustrated in Fig. 8, in order to maximize hardware resource utilization. Additionally, the PN layers are processed in the background of computing the linear layers which generate activations Y_0 and Y_1 defined in Fig. 1

B. ARCHITECTURE OF 8 × 2 BIT MM UNIT

Fig. 9(a) shows a block diagram of the 8 × 2-bit MM unit, which is used to compute the linear layers with ternary weights, as shown in Fig. 8. The ternary weights are stored in BRAM with a word width of 1024 bits. The 8-bit activations are basically stored in BRAM at 512 bits per word, whereas URAM is used for the activation (matrix) Z defined in Fig. 1 because Z is four times larger than the other matrices.

The 8 × 2-bit MM unit comprises 512 MAC units (64 columns and 8 rows). A block diagram of the MAC unit

is presented in Fig. 9(b). The MAC unit consists of eight 8 × 2-bit multipliers, an adder tree, an accumulator, and bit shifters. The circuit for the residual connections, shown in Fig. 5, is also included. For linear layers that do not include residual connections, the residual inputs are set to 0. As shown in Fig. 9(b), the 8 × 2-bit multiplier is implemented using an adder and multiplexers. Therefore, the 8 × 2-bit MM unit can be realized solely using LUTs.

C. ARCHITECTURE OF 8 × 8 BIT MM UNIT

Fig. 10(a) shows a block diagram of the 8 × 8-bit MM unit. This unit is used to compute the multi-head attention (matrix multiplication and softmax) layers, as well as the PN layers, as shown in Fig. 8. The activations (matrices) K, V, Y_0, and Y_1 defined in Fig. 1 are stored in the top BRAM, while matrix Q in Fig. 1 is divided into eight sub-matrices and stored in the left BRAMs, as shown in Fig. 7. Intermediate calculation results, such as QK^T in multi-head attention, are also stored in the left BRAMs.

The 8 × 8-bit MM unit contains 512 (64 columns and 8 rows) MAC units with INT8-limited right shifters defined in Eq. (6). These MAC units are implemented using the DPS slices (DSP48E2) available in Ultrascale+ FPGAs [26]. Although a DSP48E2 slice supports up to a 27 × 18-bit multiplier and a 48-bit accumulator, only 8 × 8-bit and 22-bit precision is utilized in this design, as shown in Fig. 10(b). As illustrated in Fig. 7, the proposed accelerator incorporates two 8 × 8-bit MM units, enabling the computation of 1024 MAC operations in a single clock cycle.

IV. FPGA IMPLEMENTATION RESULTS

As explained in Section III, the proposed accelerator for the FQ MA-BERT model was implemented on a ZCU104. The synthesis was performed using Vivado (ver. 2019.1), achieving a clock frequency of 193 MHz. PL utilization is summarized in Table 3. The theoretical peak performance of the accelerator is listed in Table 4. Each MAC unit in the 8 × 2-bit MM unit computes eight MAC operations per clock cycle, as shown in Fig. 9(b). Given that the 8 × 2-bit MM unit contains 512 MAC units, it achieves a theoretical peak performance of 1,581 GOPS. In contrast, each 8 × 8-bit MM unit consists of 512 MAC units (DSP slices), as shown in Fig. 10. Thus, two 8 × 8-bit MM units can reach 395 GOPS.

The actual (average) performance is also listed in Table 4. In this paper, the actual performance is obtained by dividing the total number of MAC operations required in the proposed FQ MA-BERT model by the corresponding computation latency. As indicated in Table 4, the actual performance is lower compared to the theoretical peak performance. This is because the 8 × 2-bit and 8 × 8-bit MM units do not operate simultaneously at all times, as shown in Fig.8, and overheads such as data movement are also required.

Table 5 shows a comparison between the proposed accelerator and conventional FPGA-based BERT accelerators [11], [12], [13], [14], [15]. Conventional accelerators require distinct data formats for nonlinear functions. However, the proposed accelerator eliminates this requirement, because the proposed FQ MA-BERT model consists of only INT-precision linear matrix arithmetic operations without the need for specialized circuits to perform nonlinear functions. The inference accuracies on the GLUE benchmark are comparable to those reported in the conventional studies. The power dissipation of the proposed accelerator is obtained from a report generated using the Vivado tool. It contains the power consumption of both the PL and PS.

As indicated in Table 5, the latency and performance of the proposed accelerator are 16.4 ms and 1.44 TOPS, which are comparable to NPE [12] and FET-OPU [13], respectively. However, the proposed accelerator requires significantly fewer hardware resources. For a fair comparison, we adopt the resource efficiency metric introduced in [14]. Figs 11(a) and (b) show the performance and throughput normalized by the number of LUTs and DSP slices, respectively. The proposed accelerator achieves over 20% higher performance per LUT and DSP compared to BAT [15] (Fig. 11(a)). Furthermore, the throughput per DSP of the proposed accelerator are 1.8 times higher than that of NPE [12] (Fig. 11(b)). These results indicate that the proposed accelerator achieves the highest performance and throughput with fewer hardware resources. This efficiency is attributed to three key characteristics of the proposed FQ MA-BERT model: 1) it is fully quantized with ternary weights, 2) it comprises only matrix arithmetic operations (i.e., it achieves a higher uniformity of calculation), and 3) it primarily relies on addition operations and does not require divide operations (i.e., it consists of easy operations). These characteristics align precisely with the requirements for the hardware-friendly models defined in Section I.

Finally, we discuss the potential for porting the FPGA-based accelerator to other platforms. The proposed FQ MA-BERT accelerator is written in Verilog HDL, except for DSP slices. DSP slices are used in the 8 × 8-bit MM unit and the MAC architecture required by the MM unit is simple, as shown in Fig. 10(b). Therefore, we conclude that porting the FPGA-based FQ MA-BERT accelerator to ASICs is feasible. In addition, the proposed FQ MA-BERT model is beneficial not only for dedicated hardware, such as FPGAs and ASICs, but also for GPUs and AI chips. For example, NVIDIA H100 GPU [27] and Google Edge TPU [28] support 8-bit matrix multiplication. However, neither supports ternary precision. Thus, the effectiveness of the proposed FQ MA-BERT model may be limited for those devices.

V. CONCLUSION

In this paper, we proposed the FQ MA-BERT model. In the proposed model, all activations and weights are quantized to INT8 and ternary precision, respectively. Additionally, nonlinear functions such as softmax and normalization layers are replaced with linear functions. This means that the proposed model consists solely of INT-precision linear matrix arithmetic operations, which meets the requirements for the hardware-friendly models. To validate its

TABLE 6. Training time of proposed Three-step QAT (RTE Task).

	Batch size	Epochs	KD *1	DA *2	Training time (min)
Step #1 [20]	32	3	No	No	0.83
Step #2 [7]	16	10	Yes	No	58.3
Step #3	32	3	Yes	Yes	155.6

(*1) KD: knowledge distillation (*2) DA: data augmentation

TABLE 7. Dependence of accuracy of RTE task on learning rates and precision in parameters of softmax and PN layers.

(a)

Learning rate	Accuracy
5×10^{-5}	66.8
2×10^{-5} (This work)	69.7
5×10^{-6}	67.5

(b)

Precision	Accuracy
Ternary (2b)	64.3
4b integer	68.6
8b integer (This work)	69.7

effectiveness, we implemented the accelerator for the proposed model on the ZCU104 evaluation board, which contains UltraScale+ XCZU7EV FPGA. The implementation results demonstrate that the proposed hardware accelerator achieves 1.8 times higher hardware resource efficiency than conventional FPGA-based BERT accelerators due to the hardware-friendliness of the proposed FQ MA-BERT model.

APPENDIX A
TRAINING DETAILS

Table 6 shows the training time for the three-step QAT described in Section II-C. All training was conducted using an NVIDIA V100 GPU. The second step takes longer than the first due to the knowledge distillation with the smaller batch size and the larger number of epochs. The training time increases further for the third step because the augmented dataset proposed in [4] was used.

Next, we evaluate hyperparameter sensitivity in the third step of the three-step QAT. Table 7 shows the dependence of the accuracy of the RTE task on learning rates and on precision in the weights of the PN layers and the two-stage linear layers that replace softmax functions. In this work, we used the learning rate of 2e-5, which was used for training the MA-BERT [7] and TernaryBERT [4] models. This learning rate is also the optimal choice in the proposed FQ MA-BERT model, as shown in Table 7 (a). Table 7 (b) indicates that as the precision of the parameters in the softmax and PN layers decreases, the accuracy deteriorates. Therefore, in this work, we used 8-bit integer precision for these layers, as illustrated in Fig. 1(c).

ACKNOWLEDGMENT

This work was partly conducted in the AI Chip Design Open Innovation Laboratory, AIST. The authors used ABCI 3.0 provided by AIST and AIST Solutions with support from "ABCI 3.0 Development Acceleration Use."

REFERENCES

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol. (NAACL)*, Jan. 2018, pp. 4171–4186.

[2] Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, and D. Zhou, "MobileBERT: A compact task-agnostic BERT for resource-limited devices," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 2158–2170.

[3] H. Fuketa and K. Uchiyama, "Edge artificial intelligence chips for the cyberphysical systems era," *Computer*, vol. 54, no. 1, pp. 84–88, Jan. 2021.

[4] W. Zhang, L. Hou, Y. Yin, L. Shang, X. Chen, X. Jiang, and Q. Liu, "TernaryBERT: Distillation-aware ultra-low bit BERT," in *Proc. Conf. Empirical Methods Natural Lanuages Process. (EMNLP)*, 2020, pp. 509–521.

[5] H. Bai, W. Zhang, L. Hou, L. Shang, J. Jin, X. Jiang, Q. Liu, M. Lyu, and I. King, "BinaryBERT: Pushing the limit of BERT quantization," in *Proc. 59th Annu. Meeting Assoc. Comput. Linguistics 11th Int. Joint Conf. Natural Lang. Process.*, 2021, pp. 4334–4348.

[6] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer, "I-BERT: Integer-only BERT quantization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Jan. 2021, pp. 5506–5518.

[7] N. W. Ming, Z. Wang, C. Liu, R. S. M. Goh, and T. Luo, "MA-BERT: Towards matrix arithmetic-only BERT inference by eliminating complex non-linear functions," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Feb. 2023, pp. 1–12.

[8] X. Tang, Y. Wang, T. Cao, L. L. Zhang, Q. Chen, D. Cai, Y. Liu, and M. Yang, "LUT-NN: Empower efficient neural network inference with centroid learning and table lookup," in *Proc. 29th Annu. Int. Conf. Mobile Comput. Netw.*, Oct. 2023, pp. 1–15.

[9] H. Fuketa, "Lookup table-based computing-in-memory macro approximating dot products without multiplications for energy-efficient CNN inference," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 10, pp. 3954–3963, Oct. 2023.

[10] H. Fuketa, T. Katashita, Y. Hori, and M. Hioki, "Multiplication-free lookup-based CNN accelerator using residual vector quantization and its FPGA implementation," *IEEE Access*, vol. 12, pp. 102470–102480, 2024.

[11] Z. Liu, G. Li, and J. Cheng, "Hardware acceleration of fully quantized BERT for efficient natural language processing," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Feb. 2021, pp. 513–516.

[12] H. Khan, A. Khan, Z. Khan, L. B. Huang, K. Wang, and L. He, "NPE: An FPGA-based overlay processor for natural language processing," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, Feb. 2021, p. 227.

[13] Y. Bai, H. Zhou, K. Zhao, H. Wang, J. Chen, J. Yu, and K. Wang, "FET-OPU: A flexible and efficient FPGA-based overlay processor for transformer networks," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–9.

[14] J. Wu, M. Song, J. Zhao, Y. Gao, J. Li, and H. K.-H. So, "TATAA: Programmable mixed-precision transformer acceleration with a transformable arithmetic architecture," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 18, no. 1, pp. 1–31, Mar. 2025.

[15] Y. Ji, C. Fang, S. Ma, H. Shao, and Z. Wang, "Co-designing binarized transformer and hardware accelerator for efficient end-to-end edge deployment," in *Proc. 43rd IEEE/ACM Int. Conf. Comput.-Aided Design*, Oct. 2024, pp. 1–9.

[16] S. Shen, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, "PowerNorm: Rethinking batch normalization in transformers," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jan. 2020, pp. 8741–8751.

[17] F. Li, B. Liu, X. Wang, B. Zhang, and J. Yan, "Ternary weight networks," 2016, *arXiv:1605.04711*.

[18] (2025). *BERT Base Model (Uncased)*. Accessed: Mar. 7, 2025. [Online]. Available: https://huggingface.co/google-bert/bert-base-uncased

[19] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *Proc. EMNLP Workshop BlackboxNLP, Analyzing Interpreting Neural Netw. (NLP)*, Apr. 2018, pp. 353–355.

[20] *Text Classification Examples, GLUE Tasks*. Accessed: Mar. 7, 2025. [Online]. Available: https://github.com/huggingface/transformers/tree/main/examples/pytorch/text-classification

[21] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.

[22] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," 2019, *arXiv:1910.01108*.

[23] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, and Q. Liu, "TinyBERT: Distilling BERT for natural language understanding," in *Proc. Findings Assoc. Comput. Linguistics: EMNLP*, 2020, pp. 4163–4174.
[24] AMD. *ZCU104 Evaluation Board User Guide*. Accessed: Mar. 7, 2025. [Online]. Available: https://docs.amd.com/v/u/en-U.S./ug1267-zcu104-eval-bd
[25] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst. (NIPS)*, vol. 30, Jun. 2017, pp. 5998–6008.
[26] AMD. *UltraScale Architecture DSP Slice User Guide*. Accessed: Mar. 7, 2025. [Online]. Available: https://docs.amd.com/v/u/en-U.S./ug579-ultrascale-dsp
[27] NVIDIA. *H100 Tensor Core GPU Specifications*. Accessed: May 27, 2025. [Online]. Available: https://www.nvidia.com/en-us/data-center/h100/
[28] Coral. *Accelerator Module Datasheet*. Accessed: May 27, 2025. [Online]. Available: https://coral.ai/docs/module/datasheet/

HIROSHI FUKETA (Member, IEEE) received the B.E. degree in electrical and electronic engineering from Kyoto University, Kyoto, Japan, in 2002, and the M.E. and Ph.D. degrees in information systems engineering from Osaka University, Osaka, Japan, in 2008 and 2010, respectively. From 2010 to 2015, he was a Research Associate with the Institute of Industrial Science, The University of Tokyo, Tokyo, Japan. Since April 2015, he has been working with the National Institute of Advanced Industrial Science and Technology (AIST), Ibaraki, Japan. His research interests include the circuit design of AI chips and cryogenic CMOS circuits for quantum computers.

Dr. Fuketa has served as a TPC Member for the IEEE International Solid-State Circuits Conference (ISSCC) from 2016 to 2018. He is a TPC Member of the IEEE Asian Solid-State Circuits Conference (A-SSCC).

TOSHIHIRO KATASHITA received the Ph.D. degree from the University of Tsukuba, in 2006. In 2006, he joined the National Institute of Advanced Industrial Science and Technology (AIST) as a fixed-term Researcher. In 2008, he joined AIST as a tenure-truck Researcher. He is currently involved in research projects on high-performance computation circuit design and on hardware security. He is a member of IEICE.

YOHEI HORI (Member, IEEE) received the B.E., M.E., and Ph.D. degrees from the University of Tsukuba, Ibaraki, Japan, in 1999, 2001, and 2004, respectively. After receiving the Ph.D. degree, he spent five years as a Postdoctoral Researcher with the National Institute of Advanced Industrial Science and Technology (AIST). He moved to Chuo University as a Research Scientist, in 2008, before returning to AIST, in 2010, as a Researcher. He is currently the Team Leader of Secure Integrated Circuit Research Team with the Semiconductor Frontier Research Institute (SFRC), AIST. His current research interests include design and architecture of reconfigurable systems, open-source EDA tools, and hardware security. He is a member of IEICE and IPSJ.

MASAKAZU HIOKI (Member, IEEE) received the B.E., M.E., and Ph.D. degrees in electronics engineering from Tohoku University, Japan, in 1998, 2000 and 2003, respectively. He joined the Electroinformatics Group, Nanoelectronics Research Institute, National Institute of Advanced Industrial Science and Technology (AIST), Japan, in 2003, working on an architecture design and a circuit design of power reconfigurable FPGA with fine-grained body biasing, called Flex Power FPGA. Since 2019, he has been the Team Leader of the AI chip Design open Innovation Laboratory (AIDL), AIST. Since 2020, he has been the Group Leader of the Advanced Integrated Circuit Research Group, Device Technology Research Institute (D-Tech), AIST. He is currently the Team Leader of the Integrated Circuit Design Research Team, Semiconductor Frontier Research Center (SFRC), AIST. His research interests include an architecture and circuit design of reconfigurable systems, low power circuit design, and non-volatile memory.